Introduction à

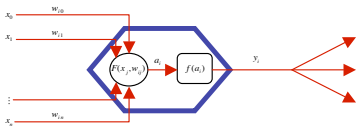
# l'Apprentissage Connexionniste

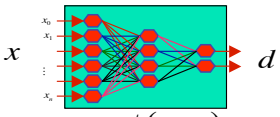
Younès BENNANI

# 2

- Rappels
- limites d'Adaline
- Madaline : Multi-Adaptive Linear Element
- Perceptron
- Théorème de convergence du Perceptron
- Capacités d'un séparateur linéaire
- Limites du Perceptron

# Rappels






$$y = \psi(x, w)$$

$$F = \{\psi(x, w) / w \in \Omega\}$$

$$E_N = \{(x^1, d^1), (x^2, d^2), \ldots, (x^N, d^N)\} / p(x,d) = p(x)p(d/x)$$

$$R(w) = \int L[d, \psi(x,w)]\, dp(x,d)$$

**Choix** →

$$\psi(x, w^*) = \underset{\psi(x,w)}{Arg\min}\, R(w)$$

**Induction** ↓

$$\tilde{R}(w) = \frac{1}{N}\sum_{k=1}^{N} L[d^k, \psi(x^k, w)]$$

**Choix** →

$$\psi(x, w^+) = \underset{\psi(x,w)}{Arg\min}\, \tilde{R}(w)$$

**Apprentissage** →

$$w(t+1) = w(t) - \frac{1}{N}\varepsilon(t)\sum_{k=1}^{N}\nabla_w \tilde{R}^k(w)$$

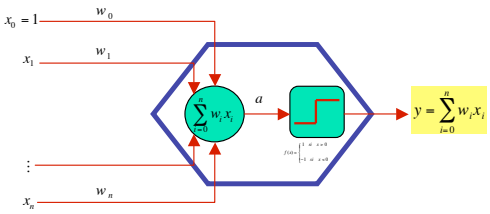
**ou**

$$w(t+1) = w(t) - \varepsilon(t)\nabla_w \tilde{R}^k(w)$$



# Rappels

**Adaline**




**Fonction de Coût**

$$\tilde{R}^k_{Adaline}(w) = \left(d^k - wx^k\right)^2$$

**Apprentissage** →

$$w(t+1) = w(t) - \varepsilon(t)\nabla_w \tilde{R}^k_{Adaline}(w)$$

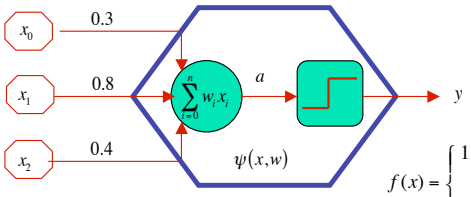
$$\nabla_w \tilde{R}^k_{Adaline}(w) = \frac{\partial \tilde{R}^k_{Adaline}(w)}{\partial w} = -2\left[d^k - wx^k\right]x^k$$

$$\Updownarrow$$

$$\begin{pmatrix} w_0(t+1) \\ w_1(t+1) \\ \vdots \\ w_n(t+1) \end{pmatrix} = \begin{pmatrix} w_0(t) \\ w_1(t) \\ \vdots \\ w_n(t) \end{pmatrix} - 2\,\varepsilon(t)\left[d^k - \begin{pmatrix} w_0(t) & w_1(t) & \ldots & w_n(t) \end{pmatrix}\begin{pmatrix} x_0^k \\ x_1^k \\ \vdots \\ x_n^k \end{pmatrix}\right]\begin{pmatrix} x_0^k \\ x_1^k \\ \vdots \\ x_n^k \end{pmatrix}$$

## Exercice à faire

$$E = \left\{ \left( \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, -1 \right); \left( \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix}, 1 \right); \left( \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix}, 1 \right); \left( \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix}, 1 \right) \right\}$$




$$f(x) = \begin{cases} 1 & si \quad x > 0 \\ -1 & si \quad x < 0 \end{cases}$$

Représenter dans un repère orthogonal l'ensemble des échantillons.

Utiliser l 'algorithme Adaline pour adapter les poids du modèle ($\varepsilon$=0.1).

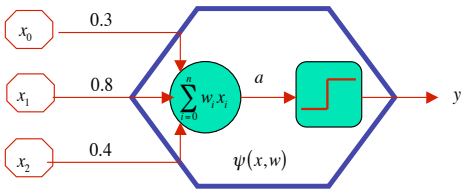
Donner l 'équation de l 'hyperplan séparant les deux classes.

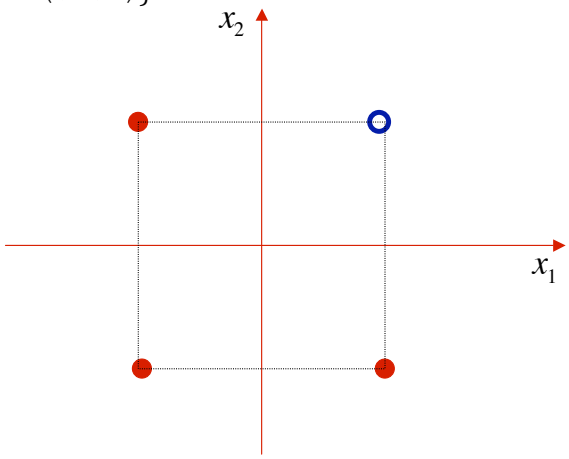
Représenter l 'hyperplan dans le même repère orthogonal



## Représentation graphique

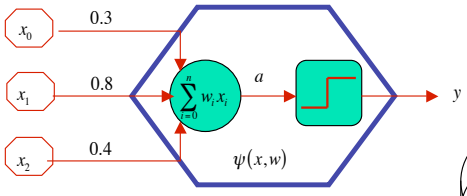
$$E = \left\{ \left( \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, -1 \right); \left( \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix}, 1 \right); \left( \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix}, 1 \right); \left( \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix}, 1 \right) \right\}$$



# Adaptation des poids

$$D = \left\{ \left( \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, -1 \right); \left( \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix}, 1 \right); \left( \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix}, 1 \right); \left( \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix}, 1 \right) \right\}$$



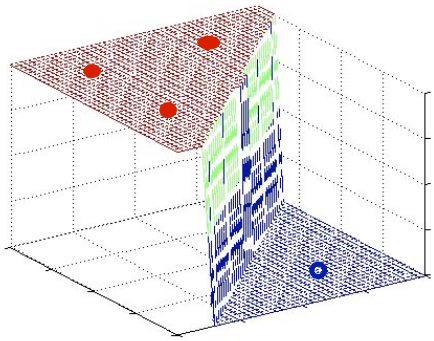

$$\begin{pmatrix} w_0(t+1) \\ w_1(t+1) \\ \vdots \\ w_n(t+1) \end{pmatrix} = \begin{pmatrix} w_0(t) \\ w_1(t) \\ \vdots \\ w_n(t) \end{pmatrix} - 2\,\varepsilon(t) \left[ d^k - \begin{pmatrix} w_0(t) & w_1(t) & \dots & w_n(t) \end{pmatrix} \begin{pmatrix} x_0^k \\ x_1^k \\ \vdots \\ x_n^k \end{pmatrix} \right] \begin{pmatrix} x_0^k \\ x_1^k \\ \vdots \\ x_n^k \end{pmatrix}$$

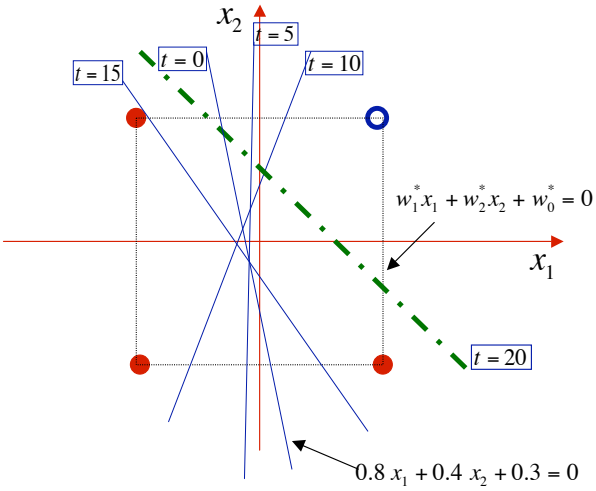
$$\begin{pmatrix} w_0(t+1) \\ w_1(t+1) \\ w_2(t+1) \end{pmatrix} = \begin{pmatrix} 0.3 \\ 0.8 \\ 0.4 \end{pmatrix} - 2 \times 0.1 \times \left[ -1 - \begin{pmatrix} 0.3 & 0.8 & 0.4 \end{pmatrix} \times \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \right] \times \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} w_0(t+1) \\ w_1(t+1) \\ w_2(t+1) \end{pmatrix} = \begin{pmatrix} 0.3 \\ 0.8 \\ 0.4 \end{pmatrix} - \begin{pmatrix} -0.5 \\ -0.5 \\ -0.5 \end{pmatrix} \longrightarrow \begin{pmatrix} w_0(t+1) \\ w_1(t+1) \\ w_2(t+1) \end{pmatrix} = \begin{pmatrix} 0.8 \\ 1.3 \\ 0.9 \end{pmatrix}$$



# Évolution de l'apprentissage

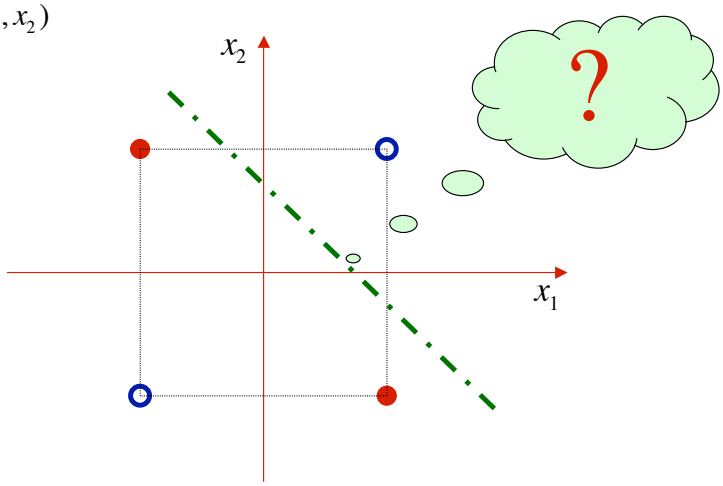




# Adaline : limites

$$\psi(x,w) = x_1 \oplus x_2 = XOR(x_1, x_2)$$

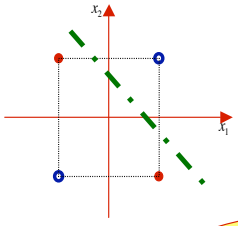
| | $x_0$ | $x_1$ | $x_2$ | $d$ |
|---|---|---|---|---|
| $x^1$ | 1 | 1 | 1 | -1 |
| $x^2$ | 1 | 1 | -1 | 1 |
| $x^3$ | 1 | -1 | 1 | 1 |
| $x^4$ | 1 | -1 | -1 | -1 |

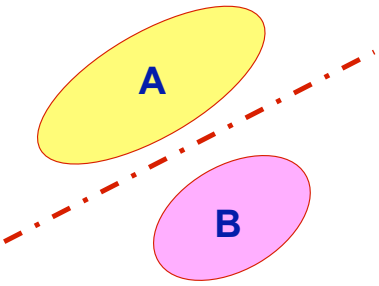




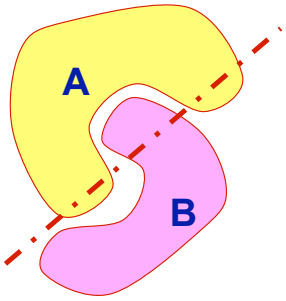

# Séparabilité linéaire

**« Deux classes d'objets, décrits dans un espace de dimension n, sont dits « linéairement séparables » s'ils se trouvent de part et d'autre d'un hyperplan dans l'espace des descripteurs»**



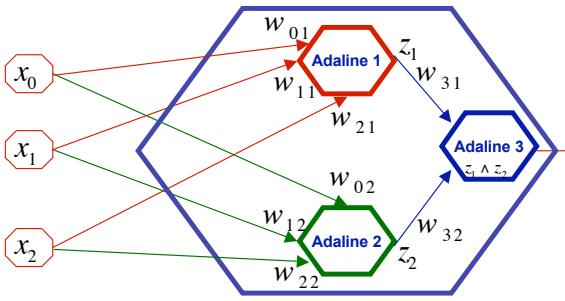

**Linéairement séparable**




**Non-linéairement séparable**

# Madaline : Multi-Adaptive Linear Element

**Madaline = un ensemble d'Adalines parallèles**



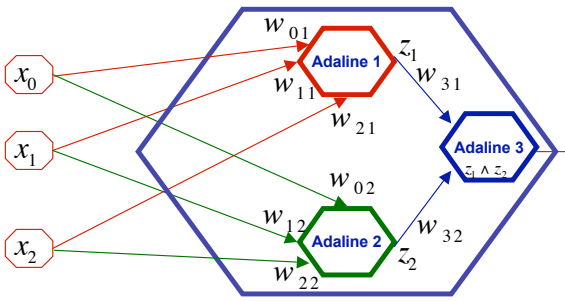

$\psi(x,w) = y = x_1 \oplus x_2$

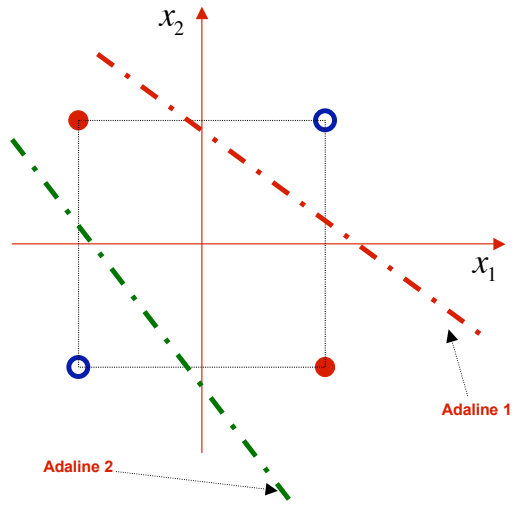
| | $x_0$ | $x_1$ | $x_2$ | Adaline 1 $d$ | Adaline 2 $d$ | Adaline 3 $d$ |
|---|---|---|---|---|---|---|
| $x^1$ | 1 | 1 | 1 | -1 ○ | 1 ● | -1 ○ |
| $x^2$ | 1 | 1 | -1 | 1 ● | 1 ● | 1 ● |
| $x^3$ | 1 | -1 | 1 | 1 ● | 1 ● | 1 ● |
| $x^4$ | 1 | -1 | -1 | 1 ● | -1 ○ | -1 ○ |



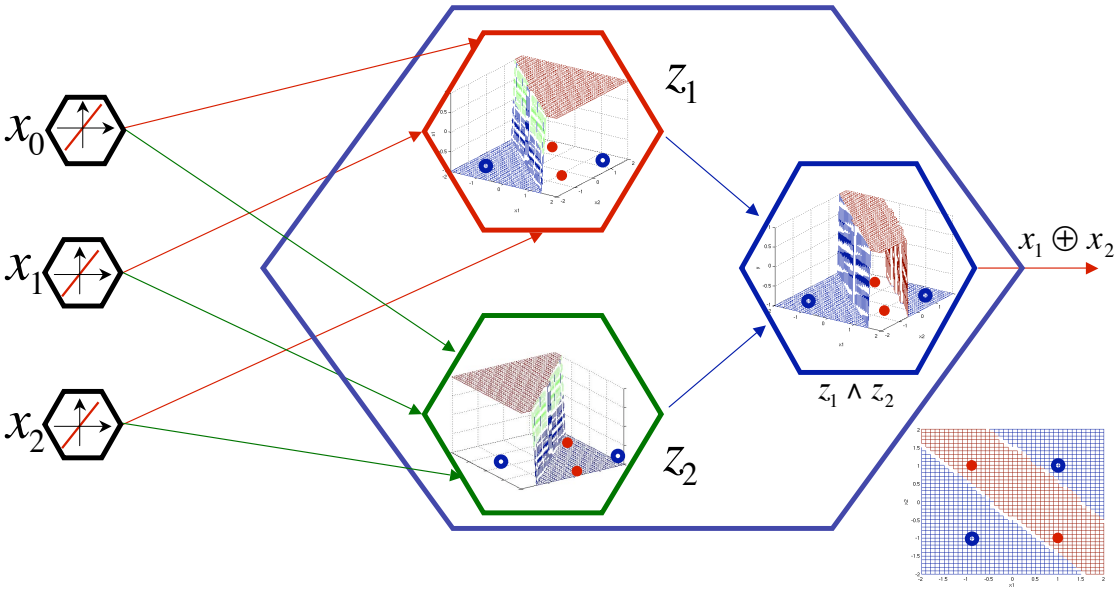
# Madaline : Multi-Adaptive Linear Element

$\psi(x,w) = y = x_1 \oplus x_2$



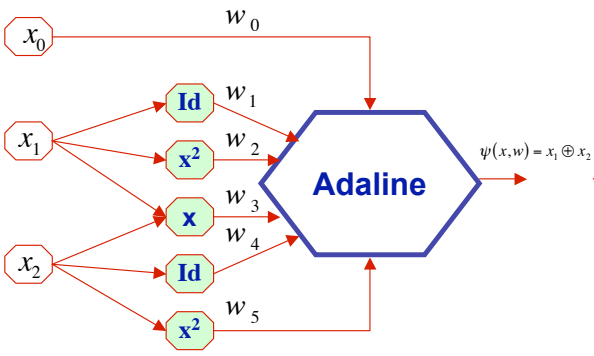
# Madaline : Multi-Adaptive Linear Element

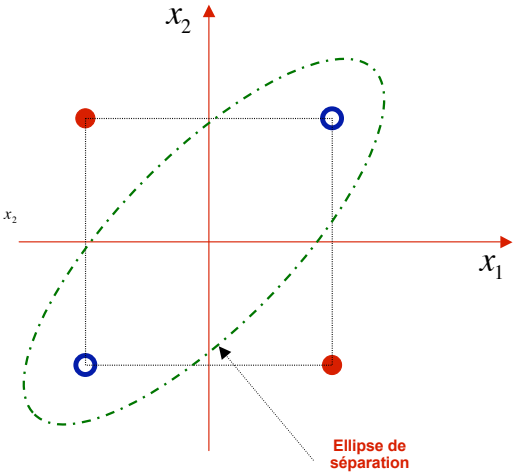





# Madaline : Multi-Adaptive Linear Element

**Adaline avec pré-traitement polynomial**




$$w_1 x_1 + w_2 x_1^2 + w_3 x_1 x_2 + w_4 x_2 + w_5 x_2^2 + w_0 = 0$$

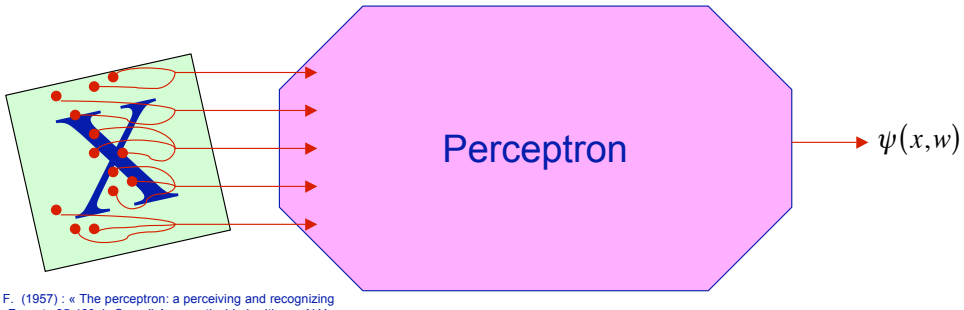
# Perceptron

Rosenblatt F., 1957, 1962*

**1957, Frank Rosenblatt**

**Le perceptron ne désigne pas un seul modèle mais regroupe une importante famille d 'algorithmes.**

**Le perceptron = machine adaptative employée pour résoudre des problèmes de classement (discrimination).**




* Rosenblatt F. (1957) : « The perceptron: a perceiving and recognizing automaton », Reports 85-460-1, Cornell Aeronautical Lab., Ithaca, N.Y.
* Rosenblatt F. (1962) : « Principles of Neurodynamics: perceptrons and theory of brain mechanisms », Spartan Books, Washington.



# Perceptron

Rosenblatt F., 1957, 1962

**la rétine $R$**

**qui reçoit les informations de l'extérieur**

**les cellules d'association**

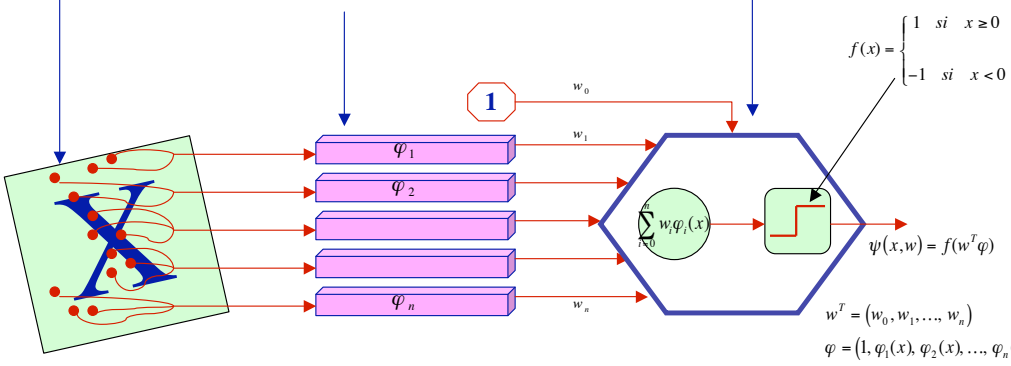
**chaque cellule possède une fonction de transition définie sur la rétine :**

$$\varphi_i(x): \quad R \rightarrow \Re$$

**la cellule de décision**

$$\psi(x,w) = f\left(w_0 + \sum_{i=1}^{n} w_i\, \varphi_i(x)\right)$$

$$f(x) = \begin{cases} 1 & si \quad x \geq 0 \\ -1 & si \quad x < 0 \end{cases}$$



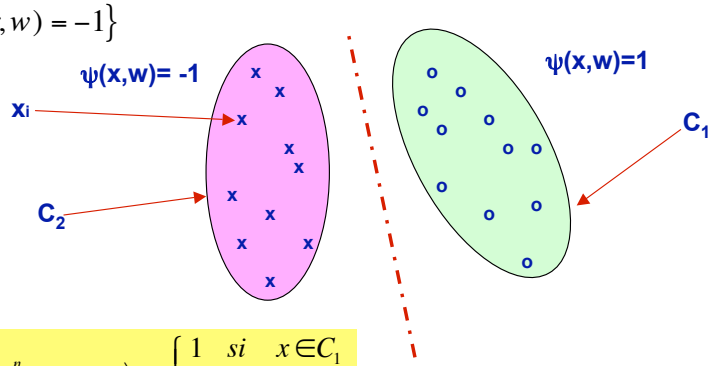
# Perceptron

## Cas de deux classes

**Un perceptron peut être vu comme un classificateur à 2 classes :**

$$C_1 = \{x \in R : \psi(x,w) = 1\}$$

$$C_2 = \{x \in R : \psi(x,w) = -1\}$$



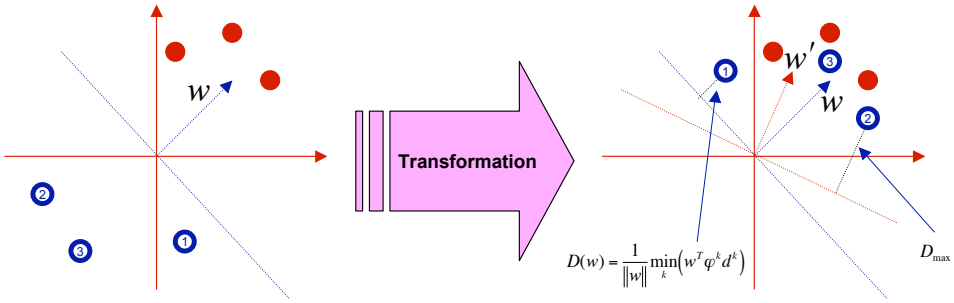

$$\psi(x,w) = f\left(w_0 + \sum_{i=1}^{n} w_i \varphi_i(x)\right) = \begin{cases} 1 & si \quad x \in C_1 \\ -1 & si \quad x \in C_2 \end{cases}$$



# Perceptron

## Cas de deux classes

$$\begin{cases} d^k = 1 & si \quad x^k \in C_1 \\ d^k = -1 & si \quad x^k \in C_2 \end{cases} \Leftrightarrow \begin{cases} w^T \varphi^k > 0 & pour \quad x^k \in C_1 \\ w^T \varphi^k < 0 & pour \quad x^k \in C_2 \end{cases} \Leftrightarrow \forall\, x^k, w^T\left(\varphi^k\, d^k\right) > 0$$




$$D_{\max} = \max_w D(w)$$

**$D_{max}$ reflète la difficulté du problème :**
**Si $D_{max}$ grand alors le problème est facile**
**Si $D_{max}$ < 0 alors pas de solution.**

# Perceptron

Cas de deux classes

**Si l 'on appelle $\left(x^k, d^k\right)$ la forme prise en compte à l'itération k,**
**On définit le carré de l'erreur instantanée associée à la forme $\left(x^k, d^k\right)$ par :**

$$\begin{cases} d^k = 1 & si \quad x^k \in C_1 \\ d^k = -1 & si \quad x^k \in C_2 \end{cases} \Rightarrow \begin{cases} w^T \varphi^k > 0 & pour \quad x^k \in C_1 \\ w^T \varphi^k < 0 & pour \quad x^k \in C_2 \end{cases} \Rightarrow \forall\, x^k, w^T\left(\varphi^k\ d^k\right) > 0$$

$$\varphi^k = \left(1, \varphi_1(x^k), \varphi_2(x^k), \ldots, \varphi_n(x^k)\right)$$

$$\Downarrow$$

$$\tilde{R}^k_{Perceptron}(w) = -\, w^T\left(\varphi^k\ d^k\right)$$

**L'erreur quadratique globale ou (MSE) est :**

$$\tilde{R}_{Perceptron}(w) = \frac{1}{N}\sum_{k=1}^{N} \tilde{R}^k_{Perceptron}(w) = -\sum_{\{k : x^k\ mal\ classé\}} w^T\left(\varphi^k\ d^k\right)$$

**Il existe plusieurs algorithmes d'apprentissage.**



# Perceptron

Cas de deux classes

**Techniques de descente de gradient (la plus grande pente) : supposons qu 'à l'instant $t$ , les poids du Perceptron soient $w(t)$ et qu 'on présente la forme $\left(x^k, d^k\right)$ , alors les poids seront modifiés par :**

$$w(t+1) = w(t) - \varepsilon(t)\nabla_w \tilde{R}^k_{Perceptron}(w)$$

**Le pas du gradient** **Le gradient instantané**

$$\nabla_w \tilde{R}^k_{Perceptron}(w) = \frac{\partial \tilde{R}^k_{Perceptron}(w)}{\partial w} = -\varphi^k\ d^k$$

$$\varphi^k = \left(1, \varphi_1(x^k), \varphi_2(x^k), \ldots, \varphi_n(x^k)\right)$$

**Critère d'arrêt = taux de classement satisfaisant.**

# Perceptron

**Algorithme d 'apprentissage : cas de 2 classes**

1- A t=0, tirer au hasard $w(0) = \left(w_0(0), w_1(0), \ldots, w_n(0)\right)^T$

2- Présenter une forme $\left(x^k, d^k\right)$

3- Calculer la sortie du perceptron et la comparer à $y^k$

$$f\left(w_0 + \sum_{i=1}^{n} \varphi_i(x)\right) = f\left(w^T \varphi^k\right)$$

4- Si $x^k$ est bien classé : $f\left(w^T \varphi^k\right) = d^k$

$$w(t+1) = w(t)$$

Si $x^k$ est mal classé : $f\left(w^T \varphi^k\right) \neq d^k$

$$w(t+1) = w(t) + \varepsilon(t)\, \varphi^k\, d^k$$

$$\begin{pmatrix} w_0(t+1) \\ w_1(t+1) \\ \vdots \\ w_n(t+1) \end{pmatrix} = \begin{pmatrix} w_0(t) \\ w_1(t) \\ \vdots \\ w_n(t) \end{pmatrix} + \varepsilon(t)\, d^k \begin{pmatrix} 1 \\ \varphi_1(x^k) \\ \vdots \\ \varphi_n(x^k) \end{pmatrix}$$

$$\varphi^k = \left(1, \varphi_1(x^k), \varphi_2(x^k), \ldots, \varphi_n(x^k)\right)$$

Où $\varepsilon(t)$ est le pas du gradient.

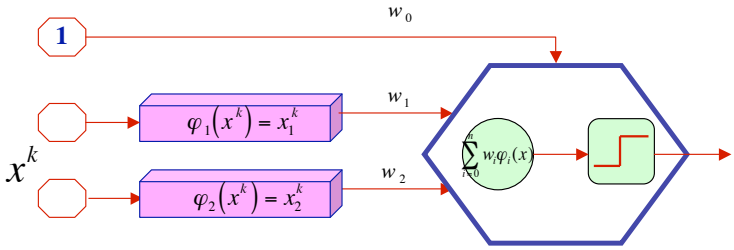
5- Répéter de 2 à 4 jusqu'à l'obtention d'une valeur acceptable de l'erreur

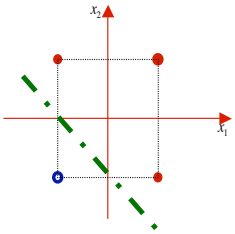


# Exemple de Perceptron

$$E_N = \left\{\left(x^1, d^1\right), \left(x^2, d^2\right), \ldots, \left(x^N, d^N\right)\right\}$$

| | $x_0$ | $x_1$ | $x_2$ | $d$ |
|---|---|---|---|---|
| $x^1$ | 1 | 1 | 1 | 1 |
| $x^2$ | 1 | 1 | -1 | 1 |
| $x^3$ | 1 | -1 | 1 | 1 |
| $x^4$ | 1 | -1 | -1 | -1 |

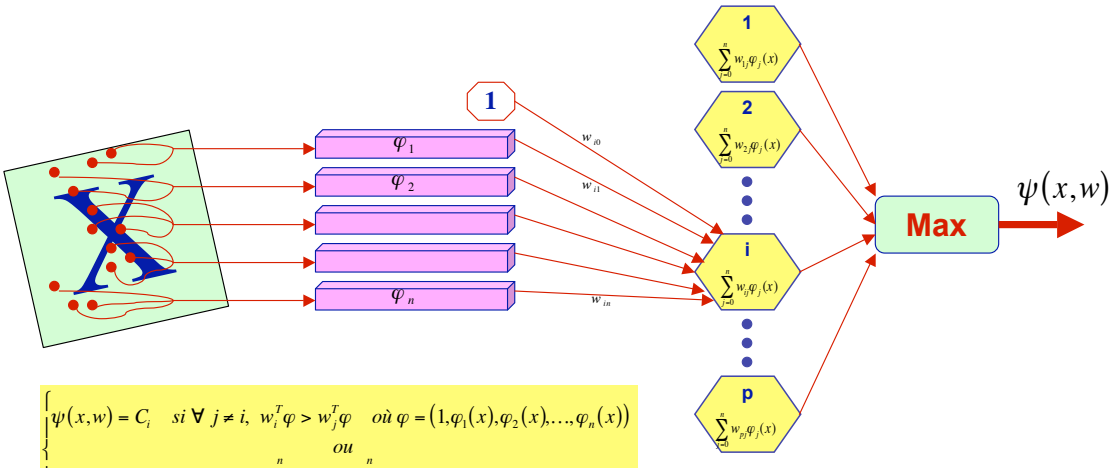



$$\psi(x, w) = x_1 \vee x_2$$

# Perceptron

Cas de p classes : $C_1$, $C_2$, …, $C_p$




$$\begin{cases} \psi(x,w) = C_i \quad si\ \forall\ j \neq i,\ w_i^T\varphi > w_j^T\varphi \quad où\ \varphi = (1,\varphi_1(x),\varphi_2(x),\ldots,\varphi_n(x)) \\ \qquad\qquad\qquad\qquad ou \\ \psi(x,w) = C_i \quad si\ \forall\ j \neq i,\ \sum_{k=1}^{n} w_{ik}\varphi_k > \sum_{k=1}^{n} w_{jk}\varphi_k \end{cases}$$



# Perceptron

**Algorithme d 'apprentissage : cas de p classes**

1- A t=0, tirer au hasard la matrice des poids $w(0)$

2- Présenter une forme $x^k \in C_i$

3- Calculer la sortie $\psi(x^k,w)$ du perceptron et la comparer à $C_i$

$$\psi(x^k,w) = C_j \quad \Leftrightarrow \quad j = Arg\max_l \left(w_l^T\ \varphi^k\right)$$

4- Si $x^k$ est bien classé : $C_j = C_i$

$$w(t+1) = w(t)$$

Si $x^k$ est mal classé : $C_j \neq C_i$

$$w_i(t+1) = w_i(t) + \varepsilon(t)\,\varphi^k$$

$$w_j(t+1) = w_j(t) - \varepsilon(t)\,\varphi^k$$

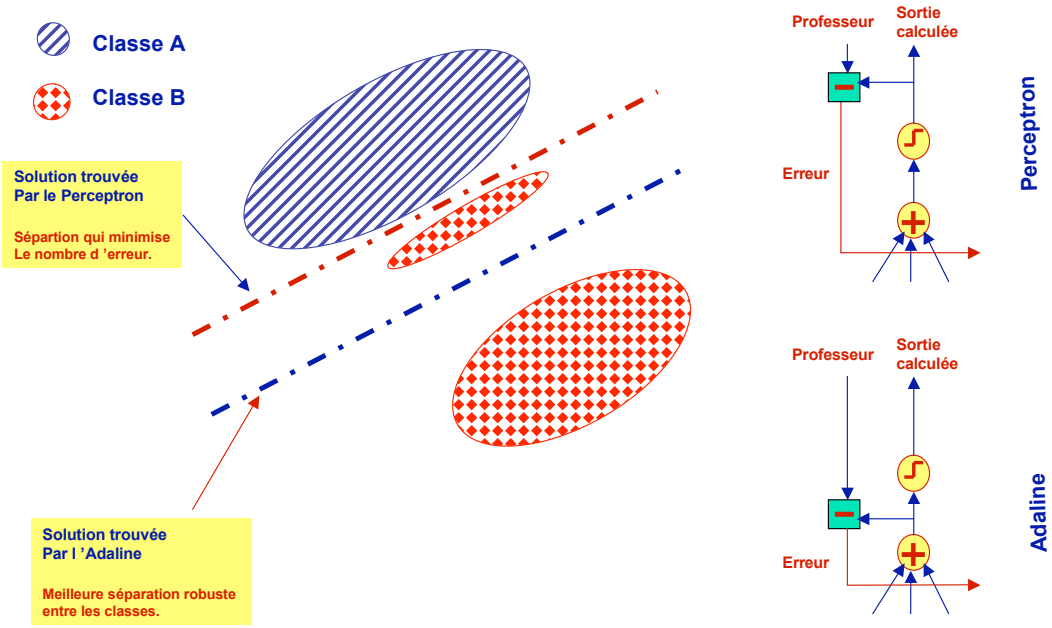
$$w_l(t+1) = w_l(t) \quad \forall\ l \neq i\ et\ l \neq j$$

Où $\varepsilon(t)$ est le pas du gradient.

5- Répéter de 2 à 4 jusqu'à l'obtention d'une valeur acceptable de l'erreur

# Perceptron et Adaline

Cas de 2 classes






# Théorème de convergence du Perceptron

***« Si un ensemble de formes est linéairement séparable, alors l'algorithme d'apprentissage du Perceptron converge vers une solution correcte en un nombre fini d'itération. »***

**Arbib M.A. (1987) : « Brains, Machines, and Mathematics »**
**Berlin, Springer-Verlag.**

**Rosenblatt F. (1962) : « Principles of Neurodynamics »**
**N.Y., Spartan.**

**Block H.D. (1962) : « The Perceptron: A Model for Brain Functioning »**
**Reviews of Modern Physics 34, 123-135.**

**Minsky M.L. & Papert S.A. (1969) : « Perceptrons »**
**Cambridge, MIT Press.**

**Diederich S. & Opper M. (1987) : « Learning of Correlated Patterns**
**Spin-Glass Networks by Local Learning Rules »**
**Physical Review letters 58, 949-952.**

# Théorème de convergence du Perceptron

**produit scalaire normalisé :**

$$\cos(\alpha)^2 = \phi = \frac{\left(w.w^*\right)^2}{\|w\|^2.\|w^*\|^2} \leq 1$$

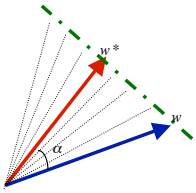
**Hypothèse :** $w^*$ : *une bonne solution*

**la règle d 'adaptation est :**

$$w(t+1) = w(t) + \varepsilon(t)\,\varphi^k\ d^k$$

**Soit** $m^k$ : *nombre d' adaptation après les présentations de* $x^k$

**après ces** $m^k$ **itérations,** $w$ **devient :** $w = \varepsilon \sum_k m^k \varphi^k d^k$




**et** $w.w^* = \varepsilon \sum_k m^k \varphi^k\ d^k.w^*$

**Soit** $m = \sum_k m^k$ **Le nombre total d'adaptation**

**Alors** $w.w^* \geq \varepsilon\, m \min_k \left(\varphi^k\ d^k.w^*\right) = \varepsilon\, m\, D\left(w^*\right)\|w^*\|$

$$D(w) = \frac{1}{\|w\|}\min_k\left(w^T\varphi^k d^k\right)$$



# Théorème de convergence du Perceptron

$$\|w(t+1)\|^2 = \|w(t)\|^2 + \varepsilon^2\|\varphi^k\|^2\left(d^k\right)^2 + \underbrace{2\varepsilon\, w(t)\,\varphi^k d^k}_{\leq 0}$$

$$\leq \|w(t)\|^2 + \varepsilon^2\|\varphi^k\|^2\left(d^k\right)^2$$

**Alors** $\Delta\|w\|^2 \leq \varepsilon^2\|\varphi^k\|^2\left(d^k\right)^2$

$$\leq \varepsilon^2 n \qquad = n \qquad = 1$$

**Après** $m$ **itérations on a donc :**

$$\|w\|^2 \leq m\ \varepsilon^2 n$$

**or**

$$w.w^* \geq \varepsilon\, m \min_k\left(\varphi^k\ d^k.w^*\right) = \varepsilon\, m\, D\left(w^*\right)\|w^*\|$$

**Alors** $1 \geq \phi \geq m\dfrac{D\left(w^*\right)^2}{n.}$

**Par conséquent :** $m \leq \dfrac{n}{D_{\max}^2}$

$$D(w) = \frac{1}{\|w\|}\min_k\left(w^T\varphi^k d^k\right)$$

$$D_{\max} = \max_w D(w)$$

**Cqfd.**

# Capacités d'un séparateur linéaire*

$C(N,d)$ = le nombre maximum de séparations linéaires possibles en 2 classes dans un ensemble de $N$ points en dimension $d$ .

Pour un ensemble de $N$ points il y a $2^N$ dichotomies possibles.

- Pour $N \leq d+1$ les $2^N$ dichotomies sont linéairement séparables
- Pour $N > d+1$ seulement $C(N,d)$ sont linéairement séparables.

- Si $N \leq d+1$ alors $C(N,d) = 2^N$
- Si $N > d+1$, on peut calculer $C(N,d)$ par récurrence :

$$C(1,d) = 2 \qquad (d \geq 1)$$

$$C(N,1) = 2\,N$$

$$C(N,d) = \sum_{k=0}^{N-1} \binom{N-1}{k} C(1,d-k) \qquad \binom{p}{q} \equiv \frac{p!}{(p-q)!\,q!}$$

* Cover T. (1965) : « Geometrical and statistical properties of linear inequalities with applications in pattern recognition »
IEEE Transactions on Electronic Computers, 14:326-334.

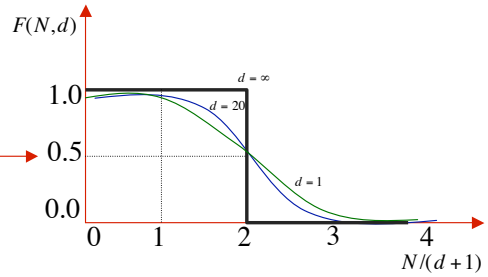


# Capacités d'un séparateur linéaire*

$F(N,d)$ = la probabilité qu'une dichotomie aléatoire soit linéairement séparable dans un ensemble de $N$ points en dimension $d$ .

$$F(N,d) = \begin{cases} 1 & si \quad N \leq d+1 \\ \dfrac{1}{2^{N-1}} \displaystyle\sum_{k=0}^{d} \binom{N-1}{k} & si \quad N > d+1 \end{cases}$$

$F(N,d) = 0.5 \quad pour \quad N = 2(d+1)$




* Cover T. (1965) : « Geometrical and statistical properties of linear inequalities with applications in pattern recognition »
IEEE Transactions on Electronic Computers, 14:326-334.

# Limites du Perceptron

**Sur les $2^{2^d}$ fonctions booléennes à $d$ entrées, moins de $\frac{2^{d^2}}{d!}$ ***

**sont réalisables par un perceptron (linéairement séparables).**

**Une limitation importante, qui a justifié à l'époque l 'abandon des recherches sur les perceptrons.**

**1969, Minsky Marvin, Papert Seymour****

**Un violent coup d'arrêt aux recherches dans le domaine**

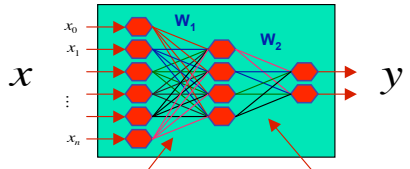
*** Lewis P.M. & Coates C.L. (1967) : « Threshold Logic » N.Y., John Wiley.**

**** Minsky M.L. & Papert S.A. (1969) : « Perceptrons » Cambridge, MA : MIT Press.**



# Architecture multi-couches

**Le « credit assignment problem »**
**On se donne un réseau en couches, et un ensemble d 'exemples composés de paires entrées-sorties.**




**On ne connaît pas les sorties Désirées des unités cachées !**

**On ne peut pas appliquer l 'algorithme d 'apprentissage du Perceptron pour déterminer $W_1$**

**Perceptron Appliquer l 'algorithme d 'apprentissage pour déterminer $W_2$**

**L 'algorithme de rétro-propagation du gradient apporte une solution d 'une simplicité déroutante à ce problème.**

# L’après pause connexionniste

**Problèmes de contrôle**

**1963, Bryson A., Denham W., Dreyfuss S.,**

**“la rétro-propagation du gradient”**

**Les années 80, la portée exacte de l ’ouvrage de Minsky & Papert sera correctement perçue.**

**La “redécouverte” de la “rétro-propagation du gradient”**

**1986, LeCun Y.**

**1986, Rumelhart D., Hinton G.E., Williams R.**


Bryson A., Denham W., Dreyfuss S. (1963) : « Optimal Programming Problem With Ineduality Constraints. I: Necessary Conditions for Extremal Solutions », AIAA Journal, Vol. 1, pp. 25-44.

LeCun Y. (1986) : « Learning Processes in Asymmetric Threshold Network »
Disordered Systems and Biological Organizations, Les Houches, France, Springer, pp. 223-240.

Rumelhart D., Hinton G.E., Williams R. (1986) : « Learning Internal Representations by Error Propagation »
In Parallel Distributed Processing: exploring the microstructure of cognition, Vol I, Badford Books, Cambridge, MA, pp. 318-362, MIT Press.